



Session 4:

DEBUG, REVIEW & IMPROVE **YOUR CODE**

What You Will Do Today



Debugged PoC

Your Session 2 PoC, with every bug found, root cause named, and a regression test locking each fix in place



Code Review Report

A severity-ranked list of issues in your own PoC, with all Critical and High items fixed and re-verified

One session. Same PoC. A completely different level of trust.

01 AI-ASSISTED DEBUGGING

You built it with Claude. Now you need to trust it.

That requires a different kind of reading.

WHEN YOU WRITE IT YOURSELF

- You know why every line exists
- You can explain your own decisions
- You know what you chose not to do
- Bugs feel familiar — they are yours

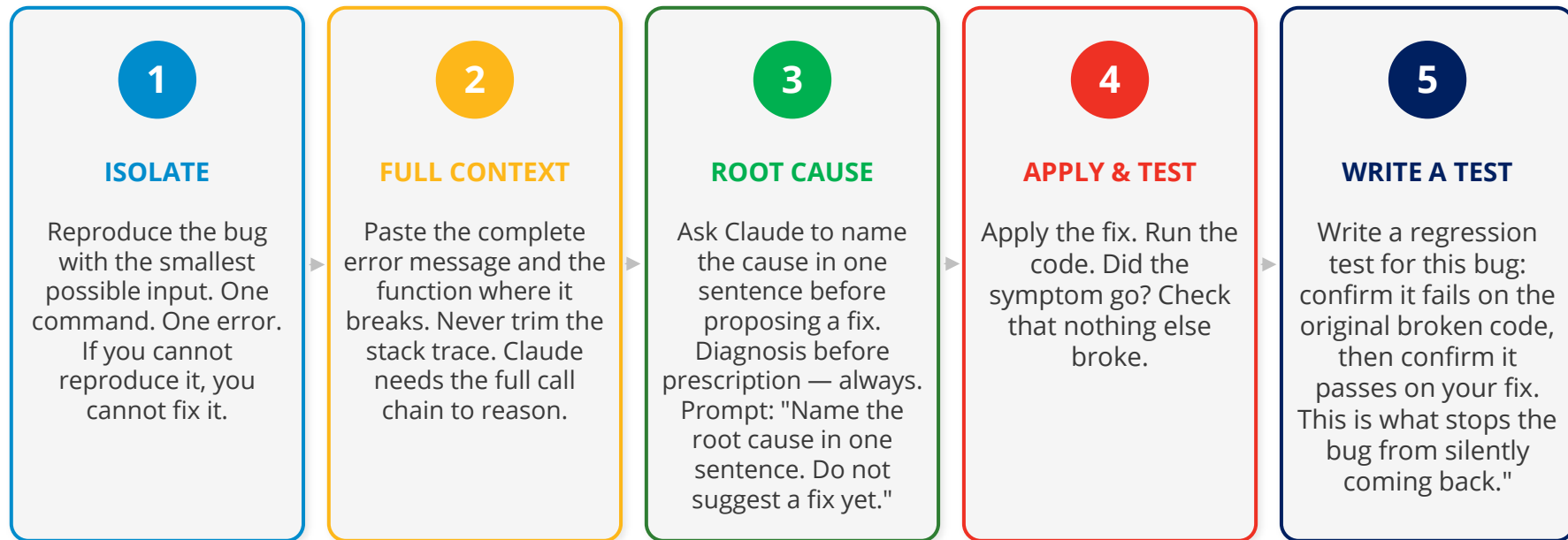
WHEN CLAUDE WRITES IT

- The code looks correct — but you did not write it
- The explanation also came from Claude
- Silent errors have no obvious signal
- Trusting it requires active verification

This is why debugging AI-generated code starts with reading — not fixing.

The Debugging Loop — Five Steps

Each pass gets you closer. Stop when the bug is fixed and a test confirms it — not before.



Step 5 is not optional. It is how a codebase gets better over time instead of trading one bug for another.

Step 1: Isolate

You cannot fix a bug you cannot reproduce on demand.

The debugging loop starts here. Before you paste anything into Claude Code, you need one command that produces one failure, reliably, every time you run it.

01 Run the smallest input

Strip the problem down. One account. One date. One API call. If you are debugging a filter that misses accounts, test it with exactly two accounts — one that should be flagged and one that should not.

02 Get a consistent failure

The failure must be reproducible. If it only happens sometimes, you do not have a bug yet — you have a symptom. Keep stripping inputs until the failure appears every single time.

03 Name what you expected vs what happened

Before you open Claude Code, write one sentence. 'I expected X. I got Y.' This is the context you will add in Step 2. Without it, you will paste an error message with no explanation — and the fix will miss.

Step 2: Full Context

Never just paste the error without explaining what you were trying to do.

Claude Code cannot see your screen. The error message alone tells it where something broke. It does not tell it what you were building, what you expected, or what the code is supposed to do. Context is what turns a generic fix into a targeted one.

✗ What most people paste

```
$ claude "fix the error"
```

- ✗ Only the error message — no context
- ✗ No explanation of what they were trying to do
- ✗ Trimmed stack trace — Claude cannot trace the call chain
- ✗ Starting a new conversation — all session context lost



✓ The format that works

```
$ claude "[full error] I was trying to [context]"
```

- ✓ **Paste the full error** — never trim the stack trace
- ✓ **Add one sentence:** what you were trying to do
- ✓ **Stay in the same session** — context builds with each pass
- ✓ **Ask Claude to diagnose** before suggesting a fix

Step 3: Root Cause

Ask Claude to name the cause in one sentence before it proposes any fix.

AI-generated code fails in specific, recognisable patterns — and they all look plausible until you test them. Knowing which pattern you are dealing with changes the fix completely.

```
$ claude "Name the root cause in one sentence.  
Do not suggest a fix yet."
```

Hallucinated library functions

Claude names a method that sounds exactly right and does not exist. The code looks correct until you run it. Diagnosis: library call fails at runtime with a 'method not found' error.

Invented API endpoints

Claude calls an endpoint that was never real. It has seen enough API patterns to construct a plausible one. Diagnosis: 404 or auth error on a call you did not write yourself.

Subtly broken logic

The code runs without error but gives the wrong answer on edge cases. No signal. Wrong output. Diagnosis: compare actual output against expected output for a known input.

Step 4: Apply & Test

Apply the fix. Run the code. Did the symptom go? Check that nothing else broke. Repeat.

The error disappearing is the beginning of review, not the end of it. A fix that makes the error message go away is a hypothesis. Running the code is what turns it into a verified fix.

1

Review before accepting

Read the diff first. Does the fix address the root cause, or does it just suppress the error message? If you cannot explain the change, do not accept it.

2

Accept and run immediately

Press Y. Run the code in the same moment. Do not accept the fix and move on without running it. The fix is unverified until the code executes.

03

Confirm the symptom is gone

The original error or wrong output should no longer appear. If it does, you are back at Step 2. Describe what is still wrong and repeat the loop.

04

Check nothing else broke

Run the full output — not just the fixed section. A fix that solves one thing and silently breaks another is not a fix. It is a trade.

If the symptom is gone and nothing else broke, go to Step 5. If not, return to Step 2 with the new failure.

Step 5: Write a Test

Prompt for a test that fails on the broken code and passes only after the fix. This is how you stop the bug from coming back.

AI-generated code has no inherent accountability. A test is the mechanism that makes it accountable. Use Claude Code to write the test — force it to prove its own fix works before you trust it.

```
$ claude "Write a test for flag_stale_accounts that fails  
if the function crashes on a missing last_contact field.  
Pass only when it handles it gracefully with a clear error."
```

Before committing the test, check:

- ✓ Does it test what it says?
- ✓ Could it pass vacuously?
- ✓ Is the expected value correct?

Unit test

Tests one function in isolation. Give it a known input and assert the exact output. If the function hallucinates a library call, the unit test catches it before it ever runs against real data.

Integration test

Tests the full flow end to end. Submit an expense, verify it is stored, verify the amount is normalised, verify the insight is generated. If any step silently fails, the integration test surfaces it.

Why this is not optional

Hallucinated functions, invented endpoints, and subtly broken logic all pass casual review. They pass because the code looks right. A test does not read the code — it runs the code. That is the difference.

Test first. Accept second. This is the order that matters.

02 AI-ASSISTED CODE REVIEW

Your PoC runs.

But would you trust it?

**Would you let your team
trust it?**

Building it was the first challenge. Standing behind it is the second.

Review Code As a Manager - Prompt

One prompt. Run it on any codebase. The output is a severity-ranked list of real issues.

THE PROMPT PATTERN

```
$ claude "Review this code as a senior engineer preparing it for a production handoff.
```

```
List every issue you find. For each issue:
```

- Name the problem specifically
- Explain why it matters
- Classify severity: Critical / High / Medium / Low

```
Do not suggest improvements or refactors. Flag problems only."
```

Senior engineer framing

Raises the bar on what Claude considers a problem worth flagging

Severity classification

Forces prioritization. You cannot fix everything in one session.

Problems only, not improvements

Prevents Claude from expanding your scope under the guise of review

Explicit list format

Produces output you can act on immediately, not a paragraph you have to parse

Pushing Claude When the First Review Is Too Gentle

The first review is rarely the best one. These prompts get you to the real issues.

Push Prompt 1 — Force severity:

When the review is all Low, name the consequence.

```
$ claude "You gave me four Low severity issues. Now find the one issue most likely to cause this PoC to fail in a live demo. Name the specific failure mode."
```

Push Prompt 2 — Target risk areas:

When the review feels generic, give it a specific lens.

```
$ claude "Review specifically for: missing input validation, hardcoded values, and unhandled exceptions. For each one, name the exact line and the consequence of leaving it unfixed."
```

Push Prompt 3 — Production lens:

When the review still feels comfortable, raise the stakes.

```
$ claude "Imagine a deployment engineer picks this up at 9am on Monday with no context. What will break by 10am? Name it specifically."
```

Use them in sequence. Each one narrows the scope and raises the stakes. Stop when the review starts finding things that actually matter.

Applying the Review — From Issues to Fixes

Not everything needs to be fixed today. The skill is deciding what does.

CRITICAL / HIGH

Fix before committing

- *Hardcoded credentials or API keys*
- *Missing input validation on user-controlled inputs*
- *Unhandled exceptions that crash the script*
- *Logic errors that produce wrong output silently*

MEDIUM

Fix before handoff

- *Hardcoded configuration values that should be arguments*
- *Missing retry logic on external calls*
- *No logging on failure paths*
- *Functions that are impossible to unit test*

LOW

Note in README

- *Variable naming*
- *Missing docstrings*
- *Code style*
- *Refactoring opportunities*

Self-Review Sprint — Your Session 2 PoC

Run the review prompt on the PoC you built. Fix the top issues. Re-run the review. Make it trustworthy.

01

Run the review prompt on your PoC

\$ claude "Review this code as a senior engineer preparing it for a production handoff. List every issue, ranked by severity."

02

Classify each finding

Critical / High / Medium / Low

03

Fix every Critical and High issue

One fix at a time. Re-run the code after each fix to confirm nothing else broke.

04

Re-run the review prompt

\$ claude "Review this code again. Did the fixes introduce any new issues?"

05

Update your README with deferred items

Note every Medium issue you are not fixing today. Do not hide them — name them as known limitations.

✓ Review run and findings classified · Critical/High issues fixed · Review re-run confirmed · README updated with deferred items

Q&A

